Repositorio del proyecto: https://github.com/donmatthiuz/RL/tree/proyecto

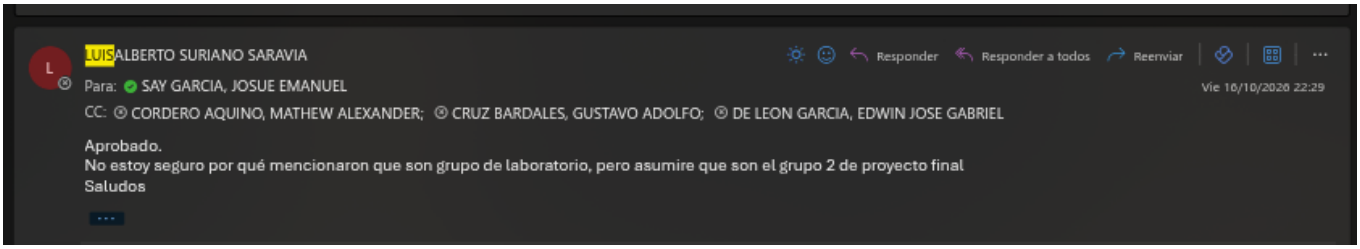
EVIDENCIA DOCUMENTAL

Constancia de entrega y aprobación

Propuesta de proyecto: Aprendizaje por Refuerzo en Polymarket

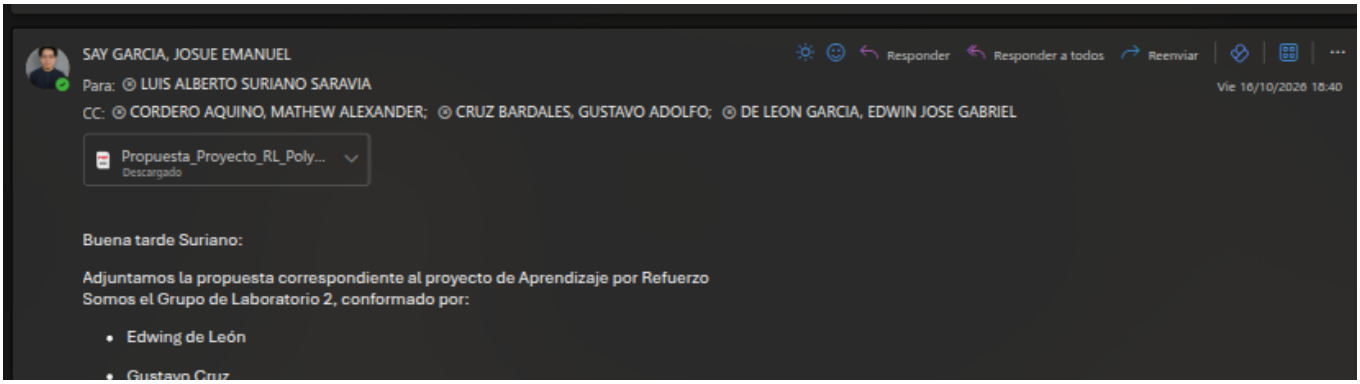
Se incorporan al inicio las capturas aportadas como evidencia de entrega oportuna de la propuesta original. En ellas constan el envío del **16 de octubre de 2026 a las 18:40** y la aprobación del profesor ese mismo día a las **22:29**.

1. Aprobación del profesor



Luis Alberto Suriano Saravia · 16/10/2026, 22:29. El mensaje confirma: «Aprobado». También indica que asume que el equipo corresponde al **grupo 2 del proyecto final**.

2. Envío de la propuesta



Jose Emanuel Say García a Luis Alberto Suriano Saravia · 16/10/2026, 18:40. La captura muestra el correo de envío, el archivo adjunto y la referencia al proyecto de Aprendizaje por Refuerzo.

> **Actualización de la propuesta: 30 de octubre de 2026.**
> Las páginas siguientes desarrollan la propuesta original e incorporan las decisiones, la documentación y el código registrados en el repositorio entre el 23 y el 30 de octubre. Las capturas se conservan sin modificaciones.

PROPUESTA ACTUALIZADA · 30 DE OCTUBRE DE 2026

PolyRL: aprendizaje por refuerzo en Polymarket

Datos históricos, decisiones secuenciales y operaciones con saldo ficticio.

Idea y objetivo

Construir un agente que observe un mercado binario histórico y aprenda cuándo **comprar YES, comprar NO, vender o esperar**. El objetivo es comprobar si su política supera a estrategias simples en mercados que no se utilizaron para entrenarlo.

Polymarket funciona como fuente de datos. El entrenamiento y la evaluación se realizarán en un simulador con **saldo ficticio**. El proyecto integra Deep Reinforcement Learning, datos reales, modelado de un entorno y evaluación estadística.

Cómo funcionará

- **1. Datos:** mercados resueltos *Bitcoin Up or Down* horarios (unos 12 500 desde mayo de 2025), con precio por minuto y alineados con el spot de Binance. ETH queda como extensión si el tiempo lo permite.
- **2. Simulación:** reproducir cada mercado como un episodio, con una posición a la vez, costos configurables y liquidación al conocerse el resultado.
- **3. Aprendizaje:** entrenar PPO y DQN interactuando con el simulador; entrenar la variante offline con un conjunto fijo de transiciones.
- **4. Comparación:** ejecutar todas las políticas sobre los mismos mercados de evaluación y medir retorno, riesgo, operaciones y costos.

Técnicas que se utilizarán

Técnica	Aplicación prevista
PPO	Agente principal; optimiza la política mediante una arquitectura actor-crítico.
DQN	Agente de comparación; estima el valor de las cuatro acciones discretas.
Offline RL	Discrete CQL; Behavior Cloning como referencia, ambos sobre transiciones fijas.
Backtesting y recompensa	Reproducción histórica y comparación de recompensas basadas en PnL, riesgo y resultado final.

> **Precisión conceptual incorporada:** usar precios históricos en un simulador no convierte por sí solo a PPO o DQN en Offline RL. Si el agente interactúa con el entorno, el entrenamiento es online. La variante offline aprende de transiciones ya recolectadas.

Resultado esperado

Una demo local en Streamlit que reproduzca un mercado paso a paso y muestre la acción, la posición y el saldo, junto con curvas de capital y una tabla comparativa frente a estrategias base. El informe explicará también los resultados negativos y las limitaciones.

DISEÑO TÉCNICO

Datos, entorno y recompensa

Decisiones basadas en una prueba con datos reales y en prácticas de proyectos open source.

Módulo	Estado y contenido
Datos	Captura del historial crudo **implementada** (comando polym4v capture). Previsto: transformación a markets.parquet, steps.parquet y manifest.json con hashes.
Entorno	Comportamiento del simulador **definido** (modelo de tiempo, reloj, costos, acciones inválidas). Previsto: PolymarketEnv propio sobre Gymnasium.
Agentes	Previsto: interfaz común para baselines, PPO, DQN y CQL/BC con stable-baselines3 y d3rlpy.
Evaluación y demo	Previsto: backtest común, métricas, intervalos de confianza y demo que reproduce decisiones guardadas.

Datos verificados con la API real

Familia elegida: **BTC Up or Down horario**, que resuelve con la vela de 1 hora de Binance; el spot de Binance reproduce el resultado. El precio por minuto se obtiene del endpoint CLOB *prices-history*: la API nueva solo guarda 1 minuto durante 7 días, por eso el historial crudo se está archivando ahora. Desde el 06/03/2026 estos mercados cobran comisión taker de **0,07 × p × (1 - p)** por acción.

Formulación del entorno

- **Episodio:** la vela de 60 minutos, con una decisión por minuto (60 pasos); liquidación a 1 o 0 al cierre, sin comisión.
- **Tiempo:** cada dato guarda cuándo ocurrió y cuándo estuvo disponible; el agente solo ve lo disponible hasta el instante de decisión.
- **Acciones:** WAIT, BUY_YES, BUY_NO y SELL; una posición a la vez. Una acción inválida se ejecuta como WAIT.
- **Costos:** medio spread de 0,01 (sensibilidad con 0,02) más la comisión oficial, iguales para todos los mercados.

> **Hallazgo que cambió el diseño:** con una grilla de 1 minuto, el precio guardado llega hasta 60 s tarde frente al spot y el agente podía "ver el futuro" (correlación de 0,90; alineando los tiempos baja a 0,22). La ejecución al precio siguiente y la latencia se definirán antes de implementar el entorno.

ORGANIZACIÓN Y PLAN DE TRABAJO

Equipo, fases y entregables

Cinco roles documentados; asignaciones tomadas de docs/03_roles.md al 30/10/2026.

Rol / integrante	Responsabilidad principal	Respaldo
R1 · Josué Líder técnico e integrador	Integración, contratos entre módulos, CLI, decisiones de diseño y coherencia de la documentación.	R4
R2 · Mathew Ingeniería de datos	APIs, dataset, features, split temporal y análisis exploratorio.	R5
R3 · Gustavo Entorno y recompensa	Simulador, cartera, costos, liquidación, invariantes y ablación de recompensa.	R4
R4 · Por asignar Agentes RL	Baselines, PPO, DQN, dataset offline, CQL/BC y entrenamiento reproducible.	R3
R5 · Por asignar Evaluación y demo	Backtesting, métricas, estadística, Streamlit, informe y presentación.	R2

Fases e hitos previstos

Fase	Hito verificable de cierre
F0 · Preparación	Validar las APIs (hecho), decidir el diseño (en curso), archivar el crudo (en curso) y completar las asignaciones.
F1 · Datos y entorno	Dataset v1, entorno validado y un baseline ejecutando episodios completos sobre datos reales.
F2 · Agentes online	PPO y DQN entrenados; prueba en entorno sintético y primera comparación en validación.
F3 · Offline y ajuste	CQL/BC, ablación de recompensa, modelos finales con cinco semillas y demo inicial.
F4 · Entrega final	Evaluación final en test, demo comparativa, informe, presentación y artefactos reproducibles.

Las fases expresan el orden del trabajo; no fijan fechas. Prioridades P0, P1 y P2: si el tiempo es limitado, se recortan primero las P2, incluida la extensión a ETH. La integración continua (CI/CD) quedó fuera del alcance.

Colaboración y criterio de finalización

El trabajo se concentra en la rama **proyecto**, con commits pequeños en formato convencional y pruebas por cada cambio de lógica. Una tarea termina cuando cumple su aceptación, tiene pruebas que pasan localmente y documentación. El uso de IA queda registrado: qué agente o herramienta se usó, para qué y quién verificó el resultado.

METODOLOGÍA Y CRITERIOS DE ÉXITO

Evaluación y calidad

El protocolo busca comprobar qué aprende el agente y bajo qué supuestos funciona.

Comparación de estrategias

Los agentes se compararán con estrategias base: **aleatoria, no operar, favorito, momentum, reversión a la media y valor justo** (calculado con el spot). Todas pasan por el mismo entorno, costos y mercados. Los modelos se eligen en validación; el test se usa una sola vez, con todo congelado.

- **Métricas:** retorno medio por mercado, retorno acumulado, Sharpe por episodio, drawdown máximo, win rate, número de operaciones, costos y porcentaje de acciones inválidas.
- **Repeticiones:** al menos cinco semillas para las configuraciones finales; media y dispersión entre semillas e intervalos de confianza del 95 % por bootstrap.
- **Análisis:** comparación pareada sobre los mismos mercados, ablación de la recompensa y, si el tiempo lo permite, sensibilidad a costos y ejecución al siguiente paso.

Hipótesis y entregables evaluables

Se estudiará si PPO supera al mejor baseline fuera de muestra, si penalizar drawdown reduce el riesgo y si CQL iguala o supera a las políticas que generaron sus datos. **No superar los baselines también será un resultado válido** si la evaluación es rigurosa y la conclusión queda sustentada.

Entregable	Verificación prevista
Dataset y entorno	Reconstrucción con el mismo hash; check_env de Gymnasium; conservación de cartera, saldo no negativo y ausencia de datos futuros.
Agentes y comparación	Checkpoints y curvas de aprendizaje para PPO, DQN y la variante offline; tabla en test con cinco semillas e IC del 95 %.
Demo e informe	Reproducción paso a paso, comparación visual y reporte de metodología, resultados, ablación y limitaciones.
Calidad y reproducibilidad	Pruebas locales (33 aprobadas hoy); versiones fijadas en uv.lock; hash del crudo y del dataset; configuración, semilla y commit por corrida.

Pruebas previstas

Pruebas con respuestas simuladas y semillas fijas. Se suman tests de no-fuga: un mercado sintético donde el precio es el valor justo, en el que ninguna política puede ganar, y la comprobación de que alterar datos futuros no cambia la observación.

Riesgos y límites del diseño

En una muestra de 298 mercados el precio ya está bien calibrado y ninguna estrategia simple gana después de costos. Con unos 1 800 mercados de test solo se detectan diferencias de 6 % o más por operación. Se fijó de antemano que, si nada gana, el resultado se reporta igual. El simulador es price-taker y aproxima **p(NO) como 1 - p(YES)**.

CAMBIOS REGISTRADOS · 30 DE OCTUBRE DE 2026

Avances y estado actual

Resumen basado en el historial de Git, los archivos del proyecto y las pruebas ejecutadas.

Commit	Cambio
23:32 · 2263a60, 01de02d	Prueba con datos reales, diseño del ETL y definición del simulador con revisión independiente.
23:32 · 497fab5	Arquitectura, tareas y decisiones (ADR-010 a ADR-013); R1 asignado; diagramas en Mermaid.
23:32 · 0dbbc65, f506afc	Paquete renombrado a **polym4v**, dependencias fijadas y código actualizado a Python vigente.
23:36 · 8d36129	Context7 como servidor MCP para escribir contra documentación actualizada.
23:40 · 0382274	Comando polym4v capture: archiva el historial crudo con reintentos, caché y hashes.

Implementado y comprobado

- **Captura del crudo:** iniciada para BTC horario y pausada a las 23:50 para este reporte, con 1 056 mercados archivados (2 111 series de precio YES y NO) y el hash de cada archivo. Se retoma sin repetir descargas.
- **Uso de IA documentado:** agentes existentes copiados sin cambios, skills del proyecto y Context7, con el para qué de cada uno.

> **Verificación local:** `uv run pytest -q`
> **Resultado: 33 pruebas aprobadas** (22 de la integración con Polymarket y 11 de la captura). El transporte HTTP está simulado.

Pendiente frente al plan

Decidir la ejecución, la latencia, la valoración de la posición y el fin del episodio; transformar el crudo en el dataset; implementar el entorno, los baselines, los agentes, el backtesting y la demo. A esta fecha no hay resultados de agentes entrenados.

Fuentes: **README.md**, **docs/01** a **docs/09**, **docs/notas/**, **pyproject.toml**, **src/polym4v/** y **tests/**.

> **Repositorio del proyecto · rama proyecto**
> https://github.com/donmatthiuz/RL/tree/proyecto